

DOCUMENTACIÓN DOCKER

PROYECTO XUXEMONS

Olaya | Ares | Deivid



Tabla de contenidos

Explicación del docker-compose.yml.....	2
Servicio db - MySQL 8.0.....	3
Servicio backend - Laravel (PHP + Apache).....	4
Servicio frontend - Angular (Node 20).....	5
Servicio scheduler - Laravel Scheduler.....	6
Volúmenes y red.....	6
Comandos básicos de uso.....	7
Pruebas de funcionamiento del despliegue.....	7
Contenedores en ejecución.....	7
Frontend accesible.....	7
Backend operativo.....	7
Base de datos con datos.....	7
Scheduler funcionando.....	7



Explicación del docker-compose.yml

El fichero *docker-compose.yml* define cuatro servicios que trabajan conjuntamente dentro de una red interna llamada *xuxemons_network*. Todos los contenedores se comunican entre ellos por esta red y ningún servicio de aplicación es accesible directamente desde el exterior excepto por los puertos expuestos.

Servicio db - MySQL 8.0

Contiene la base de datos de la aplicación. Utiliza la imagen oficial de MySQL 8.0 y expone el puerto 3306. Las credenciales (usuario, contraseña y nombre de la base de datos) se configuran mediante variables de entorno. Los datos se guardan en un volumen llamado *db_data* para que persistan entre reinicios de los contenedores. Incluye un *healthcheck* que comprueba cada 5 segundos que MySQL está operativo, de forma que los demás servicios no se inician hasta que la base de datos está lista.

```
name: Xuxemons

services:
  db:
    container_name: xuxemons_db
    image: mysql:8.0
    ports:
      - "3306:3306"
    environment:
      MYSQL_ROOT_PASSWORD: 1234
      MYSQL_DATABASE: xuxemons
      MYSQL_USER: xuxemons_user
      MYSQL_PASSWORD: 1234
    volumes:
      - db_data:/var/lib/mysql
    networks:
      - xuxemons_network
    healthcheck:
      test: [ "CMD", "mysqladmin", "ping", "-h", "localhost", "-uroot", "-p1234" ]
      interval: 5s
      timeout: 5s
      retries: 10
```



Servicio backend - Laravel (PHP + Apache)

Construido a partir del *Dockerfile* de la carpeta *backend*. Expone el puerto *8000* (mapeado al puerto *80* interno de Apache). Depende del servicio *db* y espera que esté sano (*service_healthy*) antes de arrancar. Al iniciarse ejecuta automáticamente: la instalación de dependencias con Composer, las migraciones de la base de datos con seeders (*migrate:fresh --seed*) y finalmente arranca Apache. La zona horaria se configura como Europe/Madrid para el correcto funcionamiento de las tareas programadas.

```
backend:
  container_name: xuxemons_laravel
  build: ./backend
  ports:
    - "8000:80"
  depends_on:
    db:
      condition: service_healthy
  volumes:
    - ./backend:/var/www/html
  networks:
    - xuxemons_network
  environment:
    TZ: Europe/Madrid
  command: >
    bash -c " curl -sS https://getcomposer.org/installer | php --
    --install-dir=/usr/local/bin --filename=composer && composer install &&
    php artisan migrate:fresh --seed --force && apache2-foreground "
```

```
FROM php:8.3-apache

RUN apt-get update && apt-get upgrade -y && apt-get install -y \
  git \
  unzip \
  zip \
  libzip-dev \
  cron \
```



```
&& apt-get clean \
&& rm -rf /var/lib/apt/lists/*

RUN docker-php-ext-install pdo pdo_mysql zip

RUN a2enmod rewrite

# Cambiar DocumentRoot a public
ENV APACHE_DOCUMENT_ROOT=/var/www/html/public

RUN sed -ri -e 's!/var/www/html!${APACHE_DOCUMENT_ROOT}!g' /etc/apache2/sites-available/*.conf
RUN sed -ri -e 's!/var/www/html!${APACHE_DOCUMENT_ROOT}!g' /etc/apache2/apache2.conf

WORKDIR /var/www/html
```

Servicio frontend - Angular (Node 20)

Utiliza la imagen oficial de Node 20. Monta la carpeta *frontend* como volumen y expone el puerto 4200. Al iniciarse instala las dependencias con *npm install* y arranca el servidor de desarrollo de Angular configurado para aceptar conexiones desde cualquier dirección (*--host 0.0.0.0*).

```
frontend:
  container_name: xuxemons_angular
  image: node:20
  ports:
    - "4200:4200"
  volumes:
    - ./frontend:/app
  working_dir: /app
  networks:
    - xuxemons_network
  command: >
    bash -c "npm install && npx ng serve --host 0.0.0.0 --port 4200"
```



Servicio scheduler - Laravel Scheduler

Servicio independiente basado en la misma imagen que el backend, dedicado exclusivamente a ejecutar las tareas programadas de Laravel (recompensas diarias de xuxes y Xuxemons). Ejecuta `php artisan schedule:run` cada 60 segundos en un bucle infinito, simulando el comportamiento del `cron` del sistema operativo dentro del contenedor. Depende también de que la base de datos esté operativa.

```
scheduler:
  container_name: xuxemons_scheduler
  build: ./backend
  depends_on:
    db:
      condition: service_healthy
  volumes:
    - ./backend:/var/www/html
  networks:
    - xuxemons_network
  environment:
    TZ: Europe/Madrid
  command: >
    bash -c " curl -sS https://getcomposer.org/installer | php --
    --install-dir=/usr/local/bin --filename=composer && composer install &&
    while true; do php artisan schedule:run; sleep 60; done "
```

Volúmenes y red

El volumen `db_data` garantiza que los datos de MySQL se mantengan entre reinicios. La red `xuxemons_network` de tipo `bridge` permite la comunicación interna entre servicios por el nombre del contenedor (por ejemplo, el backend se conecta a la base de datos usando `db` como host).

```
volumes:
  db_data:

networks:
  xuxemons_network:
    driver: bridge
```



Comandos básicos de uso

Función	Comando
Iniciar la aplicación	<code>docker-compose up -d --build</code>
Iniciar la aplicación (sin reconstruir las imágenes)	<code>docker-compose up -d</code>
Parar los contenedores (sin eliminar datos)	<code>docker-compose down</code>
Parar los contenedores y eliminar los datos	<code>docker-compose down -v</code>
Reconstruir las imágenes desde cero	<code>docker-compose build --no-cache</code> <code>docker-compose up -d</code>
Ver el estado de los contenedores	<code>docker-compose ps</code>
Ver los logs en tiempo real	<code>docker-compose logs -f</code>
Ver los logs de un servicio concreto	<code>docker-compose logs -f backend</code> <code>docker-compose logs -f frontend</code> <code>docker-compose logs -f scheduler</code>
Acceder al terminal de un contenedor	<code>docker-compose exec backend bash</code> <code>docker-compose exec db bash</code>
Ejecutar migraciones manualmente	<code>docker-compose exec backend php artisan migrate</code>

Pruebas de funcionamiento del despliegue

En este apartado se incluyen las capturas de pantalla que demuestran que el despliegue de los contenedores funciona correctamente.

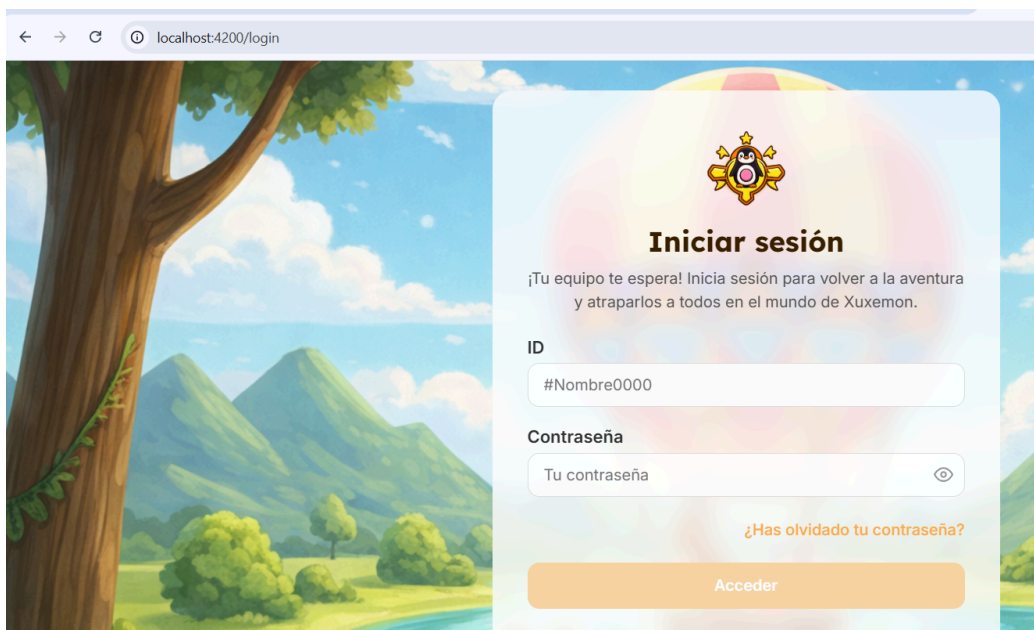


Contenedores en ejecución

  **xuxemons**
C:\Users\obaa\Desktop\obidane\Xuxemons

	frontend ● node:20 4200:4200 ↗				
	db ● mysql:8.0 3306:3306 ↗				
	backend ● xuxemons-backen 8000:80 ↗				
	scheduler ● xuxemons-schedu				

Frontend accesible





Backend operativo

HTTP Xuxemons / Usuarios / Registro

POST http://127.0.0.1:8000/api/registro

Docs Params Authorization Headers (9) Body Scripts

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ bir

```
1 {
2   "name": "Juan",
3   "surname": "Perez",
4   "email": "juan@gmail.com",
5   "password": "12345678"
6 }
```

Body Cookies Headers (9) Test Results

{ } JSON Preview Visualize

```
1 {
2   "message": "Usuario creado correctamente",
3   "public_id": "#Juan0006"
4 }
```

HTTP Xuxemons / Usuarios / Login ADMIN

POST http://127.0.0.1:8000/api/login

Docs Params Authorization Headers (9) Body Scripts

Body Cookies Headers (9) Test Results

{ } JSON Preview Visualize

```
1 {
2   "message": "Inicio de sesión correcto",
3   "token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJodHRwOi8vMTI3LjAuMC4xOjgwMDAvYXBPb2IibXVhMlR4UxNSVYzN0VxcCIiInN1YiI6IjEiLCJwcnYiOiJ0eBlxJ8MjI3bnNTDZmuACoDMZS0iCUNyTiyaX1cTVg",
4   "usuario": {
5     "id": 1,
6     "public_id": "#Admin0001",
7     "name": "Admin",
8     "surname": "Admin",
9     "email": "admin@gmail.com",
10    "created_at": "2026-04-30T07:58:01.000000Z",
11    "updated_at": "2026-04-30T07:58:01.000000Z"
12   }
13 }
```



Base de datos con datos

You can turn off this feature to get a quicker startup with -A

Database changed

mysql> SELECT * FROM users;

id	public_id	name	surname	email	password	created_at	updated_at
1	#Admin0001	Admin	Admin	admin@gmail.com	\$2y\$12\$H4bw3ezThy6A7AMpI4Fw6e8HPub7L/ZHq0.sFhb8pBf2Q1DQoEN1.	2026-04-30 09:58:01	2026-04-30 09:58:01
2	#Olaya0002	Olaya	Baa	olaya@gmail.com	\$2y\$12\$LzzAQZboa2pMgRFLyaG3bufGjAkk7/sSBP81uVaLJgT9BA932kSPO	2026-04-30 10:10:55	2026-04-30 10:10:55
3	#Laya0003	Laya	Btdane	laya@gmail.com	\$2y\$12\$HSTxu.cQ2Lng3MwKLD1RMOXFTIewLTTcuMpVpZcjQUKQOvaHHasza	2026-04-30 10:11:48	2026-04-30 10:11:48
4	#Aya0004	Aya	Babi	aya@gmail.com	\$2y\$12\$TZaH5jZDDKGztSWJosQ0GuCDU1lFr/XAjmKPbNrZtxVZ29KnAbDLS	2026-04-30 10:12:41	2026-04-30 10:12:41
5	#Ares0005	Ares	Gomez	ares@gmail.com	\$2y\$12\$K6YgZRHxb5ZUcxCicYH26eAk20t4zrWn52Jx.01J01QdLs.dMALcC	2026-04-30 11:12:08	2026-04-30 11:12:08

5 rows in set (0.00 sec)

Scheduler funcionando

2026-04-30 12:54:13 Running [Callback] 927.25ms DONE

2026-04-30 12:54:14 Running [Callback] 30.14ms DONE

2026-04-30 12:55:39 Running [Callback] 682.15ms DONE

2026-04-30 12:55:39 Running [Callback] 15.45ms DONE

2026-04-30 12:57:01 Running [Callback] 869.16ms DONE

2026-04-30 12:57:02 Running [Callback] 12.11ms DONE